

Online Exam Monitoring and Integrity Analytics Platform

Project Documentation — ExamGuard

Developed By: Mallu Neha Latha Reddy

Project Title: Development of Smart Examination Monitoring Platform with Integrity Analysis & Reporting System

1. Introduction & Executive Summary

The Online Exam Monitoring and Integrity Analytics Platform (ExamGuard) is an advanced, AI-powered virtual proctoring solution engineered to maintain absolute academic fairness during remote assessments. Developed using Python, Flask, OpenCV, and SQLite, the application acts as an intelligent virtual supervisor capable of autonomously tracking candidates, logging procedural violations, and delivering deep analytical insights to examiners.

2. Problem Statement & Objectives

Problem Statement

Conducting remote examinations securely without physical invigilation creates severe vulnerabilities, such as:

- Identity impersonation and proxy writing.
- Unauthorized resource browsing, window switching, and tab-navigation.
- High oversight overhead and a lack of real-time alert systems for examiners to audit candidate behavior efficiently.

Primary Objectives

- **Automated Supervision:** Utilize computer vision to continuously verify candidate presence and detect visual anomalies.
- **Violation Tracking:** Seamlessly record procedural infractions like browser focus loss, minimization, and tab switches.
- **Analytics & Scoring:** Compute real-time trust scores and generate comprehensive graphical data summaries to streamline administrative audits.

3. System Architecture & Technology Stack

The platform is designed around a modular web application architecture utilizing industry-standard technologies:

- **Backend Framework (Python & Flask):** Manages HTTP routing, server-side business logic, and secure user session management.
- **Computer Vision & AI (OpenCV):** Leverages Haar Cascade classifiers to execute real-time frame processing and facial feature tracking via webcam inputs.
- **Data Management (SQLite):** Acts as a lightweight relational database ensuring secure, persistent storage of candidate profiles, chronological event logs, and session scores.
- **Analytics & Machine Learning (Matplotlib & Scikit-learn):** Generates graphical performance trends, evaluation reports, and handles automated anomaly pattern processing.
- **Data Simulation Scripts (synthetic_data_generator.py):** Utilized for stress-testing system telemetry and scaling data pipelines.

4. System Workflow & Data Flow Diagram

The operational workflow of the ExamGuard platform follows a structured lifecycle from candidate authentication to post-exam administrative auditing:

```
[ Candidate Browser / Webcam ]
    |
    ▼ (Webcam Stream & Tab Events)
[ Flask Backend Server ('app.py') ]
    ├── [ OpenCV Computer Vision Engine ] → (Face Tracking & Anomalies)
    ├── [ Event Logger & Violation Tracker ] → (Tab-Switch & Focus Loss)
    └── [ SQLite Database ('exam_monitor.db') ] → (Session & Logs Storage)
    |
    ▼ (Data Processing & Metrics)
[ Matplotlib Analytics Module ] → [ Admin Dashboard Reports ('admin.html') ]
```

Workflow Breakdown

- **Authentication & Session Initiation:** Candidates log into the platform (`login.html`), where the system initializes an active session tracked via SQLite (`exam_monitor.db`).
- **Real-time Proctoring Loop:** During the exam (`exam.html`), the client-side interface streams webcam feeds to the backend, where OpenCV processes frames to check for valid facial presence.
- **Violation Event Logging:** If a candidate switches browser tabs or loses focus, event loggers instantly capture timestamps and log the infraction type.
- **Post-Exam Analytics Generation:** Upon exam completion, the analytics module aggregates penalties to calculate final integrity scores, rendering visual performance charts via Matplotlib for review on the admin panel (`admin.html`).

5. Core Functional Modules

- **Authentication & Authorization Module:** Provides secure login and registration portals segregated for students and administrative examiners.
- **Virtual Proctoring & Vision Engine (`ai_agent.py` & `analytics.py`):** Continuously tracks webcam streams during exams, actively flagging instances where a candidate's face is missing or multiple individuals are detected.
- **Violation & Tab-Switch Logger:** Monitors browser-level focus shifts, automatically recording precise timestamps and infraction categories when a candidate navigates away.
- **Integrity Scoring Engine:** Computes a dynamic trust index by aggregating weighted penalties derived from visual anomalies and browser violations.
- **Administrative Dashboard & Analytics (`dashboard.py`):** Empowers examiners with graphical summaries, individual student audit trails, and data visualizations rendered through Matplotlib plots.

6. Software Testing & Quality Assurance

To ensure absolute system stability, the project is backed by structured quality assurance artifacts located within the documentation directory:

- **Unit Testing (`Unit_Test_Plan_v0.1.xlsx`):** Outlines test cases, functional inputs, and expected outcomes to verify core module reliability.
- **Defect Tracking (`Defect_Tracker_Template_v0.1.xlsx`):** Records software bugs, severity ratings, reproduction steps, and resolution statuses identified during testing phases.
- **Agile Project Management (`Agile_Template_v0.1 (1).xlsx`):** Tracks sprint cycles, user stories, task backlogs, and milestone progress to ensure systematic and iterative project execution.

7. Conclusion

ExamGuard successfully bridges the gap between remote learning flexibility and rigorous examination security. By automating proctoring workflows through computer vision and structured logging, the platform ensures a fair, scalable, and transparent evaluation environment.